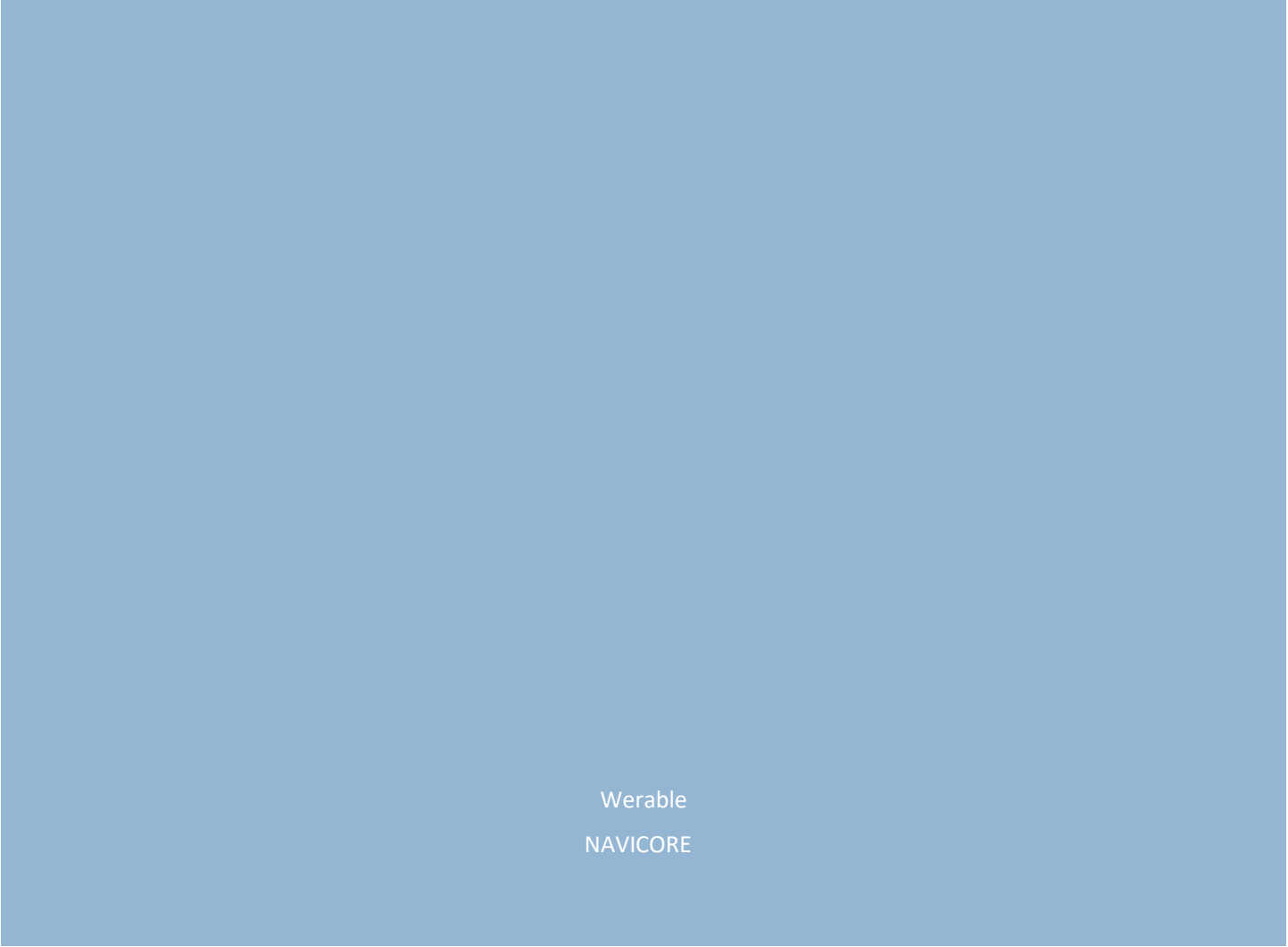




BUILD



Werable
NAVICORE

1. Stack tecnológico

Componente	Detalle
Lenguaje	Kotlin 2.2.10
UI	Jetpack Compose for Wear OS (androidx.wear.compose, NO Material3 de mobile)
Navegación	androidx.wear.compose.navigation (SwipeDismissableNavHost)
Red	Retrofit 2.11.0 + OkHttp 4.12.0 + Moshi 1.15.1 (serialización JSON, codegen vía KSP)
Concurrencia	Kotlin Coroutines 1.8.1
Persistencia	androidx.security:security-crypto (token cifrado en DataStore)
Ubicación	Google Play Services Location 21.3.0
Trabajo en segundo plano	WorkManager 2.9.1 + Foreground Service (modo “explorar activo”)
Build system	Gradle 9.4.1 (wrapper) + Android Gradle Plugin 9.2.1
Procesamiento de anotaciones	KSP 2.1.20-1.0.32 (Moshi codegen)

2. Requisitos previos

Requisito	Detalle
Android Studio	Versión reciente compatible con AGP 9.2.1 (Ladybug o superior recomendado)
JDK	21 — el proyecto usa gradle-daemon-jvm.properties con toolchainVersion=21, aprovisionado automáticamente por Gradle vía Foojay si no está instalado
Android SDK	Platform 35 (compileSdk / targetSdk), API mínima 30 (minSdk)
Conexión a internet	Requerida la primera vez, para que Gradle descargue el wrapper, el JDK toolchain y las dependencias

Nota: No hace falta instalar Gradle manualmente: el proyecto incluye el Gradle Wrapper (gradlew / gradlew.bat), que descarga automáticamente la versión 9.4.1 la primera vez que se ejecuta.

3. Obtener el código fuente

1. Extrae el archivo comprimido del proyecto (TraveXperience.rar / .zip) en una carpeta local.
2. Abre Android Studio → Open → selecciona la carpeta raíz del proyecto (la que contiene settings.gradle.kts).
3. Espera a que Android Studio detecte el proyecto Gradle e inicie la sincronización automáticamente.

4. Estructura del proyecto

Carpeta / archivo	Contenido
MainActivity.kt	Punto de entrada. Define la navegación entre Pairing / Itinerario / Nearby / Cuenta según haya sesión activa
TraveXperienceWearApp.kt	Application class: inicializa dependencias globales (TokenStore, servicios de red)
data/local/	TokenStore — persistencia cifrada de la sesión del reloj
data/network/	Configuración de Retrofit/OkHttp, interceptores de autenticación y refresh de token
data/repository/	Repositorios: PairingRepository, WearableRepository
ui/screens/	Pantallas Compose: PairingScreen, ItineraryScreen, NearbyScreen, AccountScreen
ui/viewmodel/	ViewModels con StateFlow por pantalla
ui/components/	Componentes reutilizables (mini-mapa, indicadores de estado)
service/	ExploreLocationService — Foreground Service de ubicación en segundo plano
util/	Clases de resultado (UiResult, PairingResult)

5. Sincronizar Gradle

Al abrir el proyecto, Android Studio ejecuta automáticamente una sincronización de Gradle. Si necesitas forzarla manualmente:

```
./gradlew --refresh-dependencies
```

Nota: La primera sincronización puede tardar varios minutos: descarga el wrapper de Gradle 9.4.1, el JDK 21 (si no está ya instalado) y todas las dependencias declaradas en app/build.gradle.kts.

6. Compilar el proyecto

6.1 Build de depuración (debug)

```
./gradlew assembleDebug
```

El APK resultante queda en:

app/build/outputs/apk/debug/app-debug.apk

6.2 Build de producción (release)

```
./gradlew assembleRelease
```

El APK resultante queda en:

app/build/outputs/apk/release/app-release.apk

Nota: El build type debug apunta por defecto a una IP local de desarrollo (ver Manual de Despliegue, sección 4). El build type release apunta a <https://api.travexperience.com/api/v1/>. Ajusta API_BASE_URL en app/build.gradle.kts según tu entorno antes de compilar.

7. Firma del build de release

El build type release ya tiene habilitada la minificación y ProGuard:

```
release {  
    isMinifyEnabled = true  
    proguardFiles(getDefaultProguardFile("proguard-android-optimize.txt"), "proguard-rules.pro")  
}
```

Para publicar el APK (Play Store o distribución directa), es necesario firmarlo con un keystore propio.

Pasos generales:

4. En Android Studio: Build → Generate Signed Bundle / APK.
5. Selecciona APK (o Android App Bundle si vas a subir a Play Store).
6. Crea un nuevo keystore o selecciona uno existente, e ingresa sus contraseñas.
7. Selecciona la variante release y finaliza — Android Studio genera el APK/AAB firmado.
- 8.

Nota: Guarda el keystore y sus contraseñas en un lugar seguro fuera del control de versiones: sin él no podrás publicar actualizaciones futuras de la misma app en Play Store.

8. Ejecutar la app

8.1 Emulador de Wear OS

9. En Android Studio: Tools → Device Manager → Create Device → selecciona una plantilla de reloj (categoría Wear OS).
10. Descarga una imagen de sistema Wear OS 3+ si aún no la tienes.
11. Inicia el emulador y presiona Run con el módulo app seleccionado.

8.2 Reloj físico

12. En el reloj: Ajustes → Acerca de → toca 7 veces el número de compilación para activar Opciones de desarrollador.
13. Activa Depuración ADB (y Depuración ADB por WiFi si vas a conectar sin cable).
14. Conecta por USB o WiFi (adb connect <IP-DEL-RELOJ>:5555), y selecciona el dispositivo en Android Studio antes de presionar Run.

9. Notas sobre warnings de compilación

Al compilar con Gradle 9.4.1 y AGP 9.2.1 es normal ver advertencias de deprecación (no errores) provenientes de las propias APIs internas del Android Gradle Plugin — por ejemplo, sobre declaración de dependencias con notación multi-string, que Google migrará en Gradle 10. Estas advertencias no afectan el build actual y no requieren acción del equipo de desarrollo; se resolverán con futuras actualizaciones del propio plugin de Android.